

Object Oriented Software Engineering

Chapter Four

System Analysis

Outlines

- Overview of analysis
- Analysis concepts
 - Entity, Boundary, and Control objects
 - Generalization and Specialization
- Analysis Activities: From Use Cases to Objects
 - Identifying Entity Objects
 - Identifying Boundary Objects
 - Identifying Control Objects
 - Modeling Interactions between Objects: Sequence Diagrams
 - Identifying Associations
 - Identifying Attributes
 - Reviewing the Analysis Model

An Overview of Analysis

- Analysis focuses **on producing a model of a system** that aims to be correct, complete, consistent, and verifiable.
- Analysis emphasizes on **investigation of the problem and requirements**, rather than a solution
- Translating a system specification into a **formal or semiformal model** forces developers to identify and resolve **difficult issues early in the development**.
- Developers **correct and clarify system specification** if any errors or ambiguities are found.
- The client and user are usually involved in this activity, especially **when system specification needs to be changed and when additional information needs to be gathered**.

An Overview of Analysis

- Developers **validate, correct and clarify** the requirements specification **if any errors or ambiguities are found**
- In this session, we discuss the analysis activities in **more detail**:
 - We focus on the **identification of objects**, their **behavior**, their **relationships**, their **classification**, and their **organization**
- In **object-oriented analysis**, developers build a model describing application domain.
- *The analysis model is then extended to describe how the actors and system interact to manipulate the application domain model.*

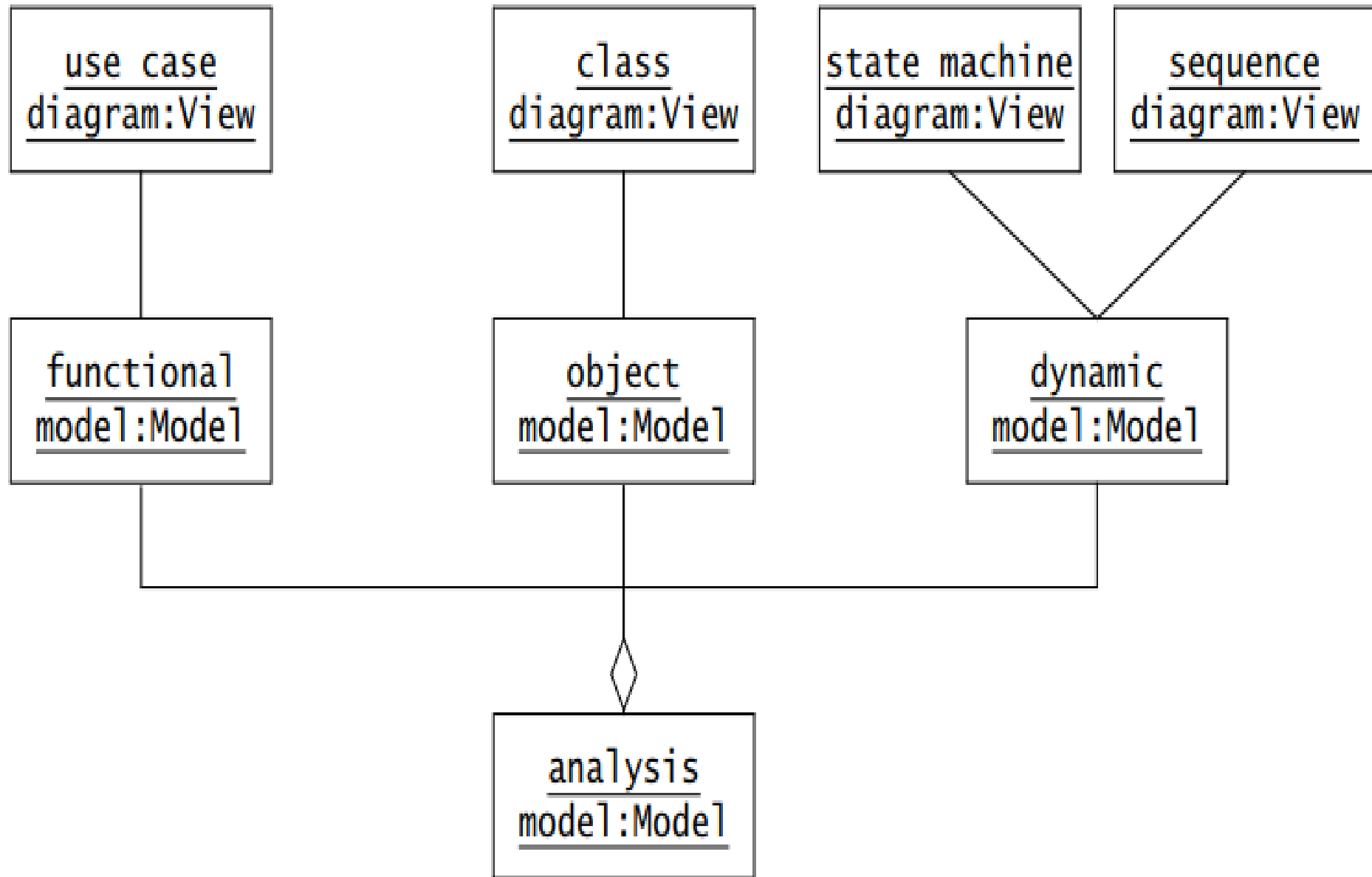
An Overview of Analysis

- *Developers use analysis model, together with nonfunctional requirements, to prepare for architecture of system developed during high-level design (**System Design**).*
- *Once developers identify problems with specification, they address them by eliciting more information from **users and client**.*
- *Requirements elicitation and analysis are **iterative** and **incremental activities** that occur **concurrently**.*

An Overview of Analysis

- The analysis model is composed of **3 individual models**:
 - Functional model, represented by **use cases and scenarios**,
 - Object model, represented by **class and object diagrams**, and
 - Dynamic model, represented **by state chart and sequence diagrams**.
- Analysis model refine **functional model** and derive **object and dynamic model**.
 - This leads to a **more precise and complete specification** as details are added to analysis model.

An Overview of Analysis



Analysis Concepts

- **The main analysis concepts**
 - Analysis Object Models and Dynamic Models
 - Entity, Boundary, and Control Objects
 - Generalization and Specialization

Analysis Concepts

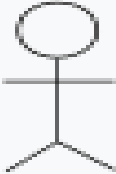
- The **analysis model** represents the system under development from the **user's point of view**
- The analysis **object** model focuses on the individual concepts that are manipulated by the **system, their properties and their relationships**
 - It is **depicted with UML class** diagrams, includes classes, attributes, and operations
 - It is a **visual dictionary** of the main concepts visible to the user
- The **dynamic model** focuses on the **behavior** of the system
 - It is depicted with **sequence diagrams** and with **statecharts**
 - Sequence diagrams represent the **interactions among a set of objects during a single use case**
 - State charts represent the **behavior of a single object** (or a group of very tightly coupled objects)

Analysis Concepts

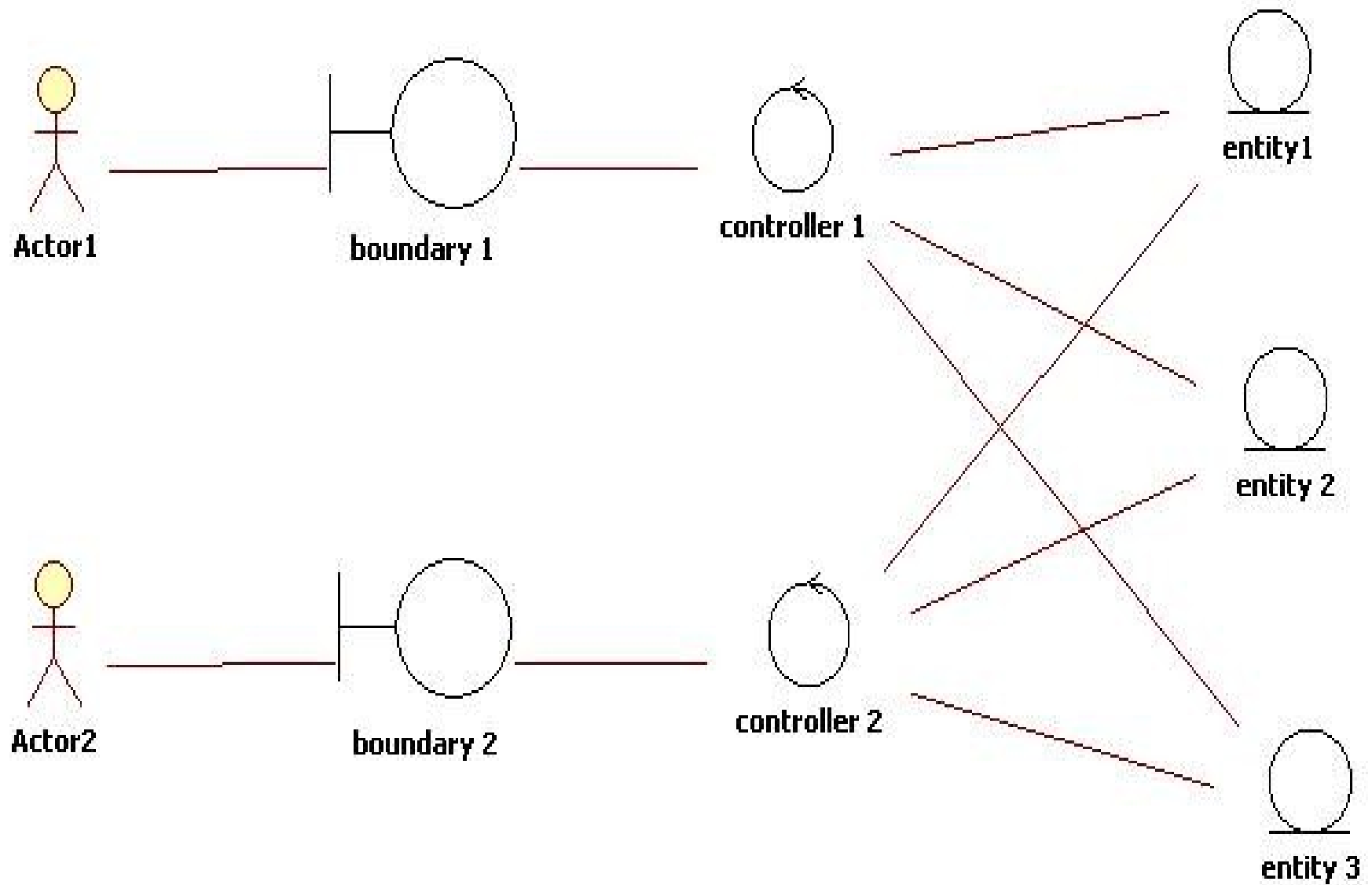
- The analysis object model consists of **entity, boundary**, and **control** objects
 - **Entity** objects: represent the **persistent information** tracked by the system
 - **Boundary** objects: represent the **interactions between the actors and the system**
 - Objects that **interface** with system actors
 - Windows, screens and menus
 - Represent the **interactions** between the actors and the system
 - **Control** objects: responsible for **coordinating** boundary and entity objects and are in charge of **realizing** use cases
 - **Mediate** between boundary and entity object
 - These serve as a **glue** between boundary elements and entity elements
 - **Implement the logic** required to manage various elements and their interactions

Robustness diagrams

Robustness diagrams allow to visually represent the relation between **entities, controls, boundaries and actors**

Representation		Relation with			
Role	Symbol	Actor	Boundary	Control	Entity
Actor		Yes	Yes	No	No
Boundary		Yes	Part/whole	Yes	No
Control		No	Yes	Yes	Yes
Entity		No	No	Yes	Yes

Analysis Concepts



Analysis Concepts

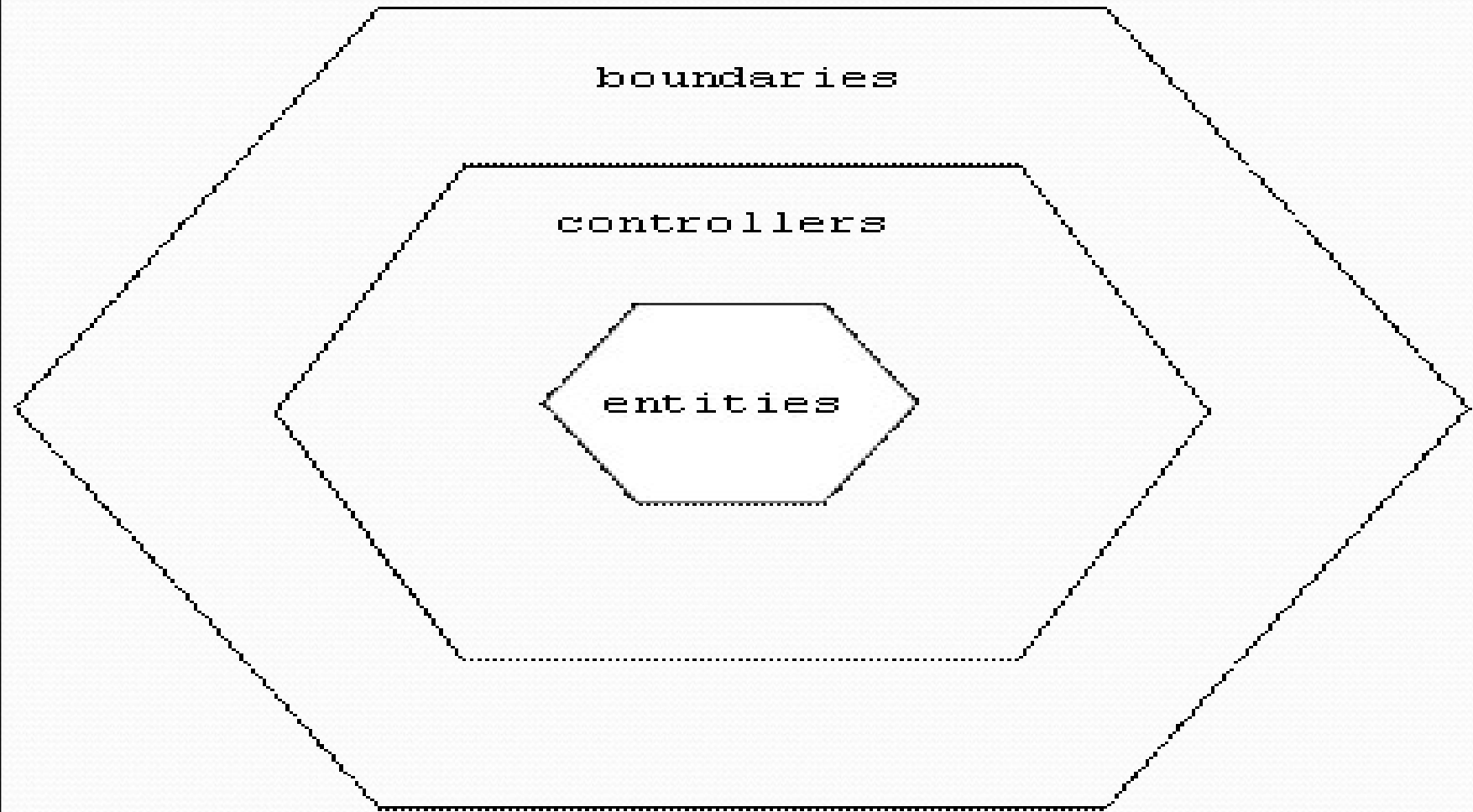
Hexagonal Architecture Pattern

actors

boundaries

controllers

entities

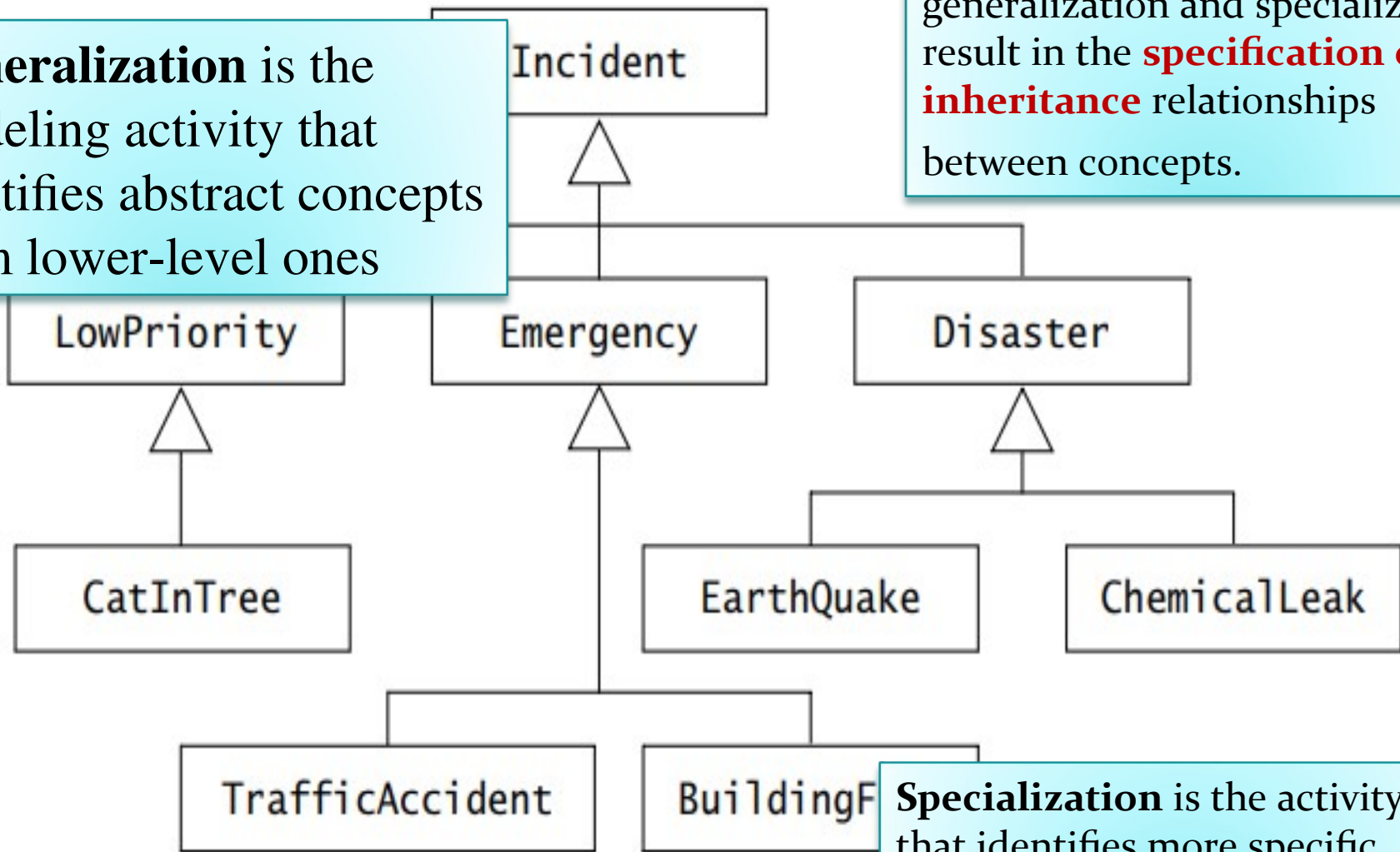


Analysis Concepts

Generalization and Specialization

Generalization is the modeling activity that identifies abstract concepts from lower-level ones

generalization and specialization result in the **specification of inheritance** relationships between concepts.



Specialization is the activity that identifies more specific concepts from a high-level one

Analysis Activities: From Use Cases to Objects

- *Analysis Activities that transform the use cases and scenarios produced during requirements elicitation into an analysis model.*
 - *Identifying entity objects*
 - *Identifying boundary objects*
 - *Identifying control objects*
 - *Mapping use cases to objects*
 - *Identifying associations among objects*
 - *Identifying object attributes*
 - *Modeling nontrivial behavior with state charts*
 - *Modeling generalization relationships*
 - *Reviewing analysis model*

Identifying entity objects

- **Entity objects** (instances of entity classes) are used to **hold and update information about some phenomenon**, such as **an event, a person, or some real-life object**..
- They are usually **persistent, having attributes and relationships** needed for **a long period, sometimes for the life of the system**.
- An entity object is usually
 - Not specific to one analysis use-case realization;
 - Not even specific to the system itself.
- The values of its attributes and relationships are often given by an actor.
- An entity object may also be needed to help perform internal system tasks.

Identifying entity objects

- Entity objects are independent of the environment (the actors).
- *Abbott's heuristics maps parts of speech to model components.*
 - *Nouns(objects), doing verbs(operations),*
 - *Having verbs(aggregation), being verbs(inheritance),*
 - *Commonnoun(classes),adjectives(attributes)*

Identifying entity objects

- **Heuristics for identifying entity objects**
 - Terms that **developers or users** need to clarify in order to understand use case
 - **Recurring** nouns in use cases
 - Real-world entities that system needs to keep track of
 - Real-world activities that system needs to keep track of
 - Use cases
 - Data sources or sinks (e.g., Printer)
 - Always use user's terms

Identifying boundary objects

- Boundary objects represent the *system interface* with the actors.
- In each use case, each actor interacts with at *least one boundary object*.
- The boundary object collects the *information from the actor* and translates it into *an interface neutral form* that can be used by *entity objects* and also by *control objects*.
- Boundary objects model the user interface at a coarse level.

Identifying boundary objects

- They **do not describe** in detail the visual aspects **of user interface**.
- For example, boundary objects such as “**button**” or “**menu item**” are too detailed.
- First, developers can discuss **user interface details** more easily with **sketches and mock-ups**.
- Second, design of **user interface design** will continue to evolve as a consequence of usability tests, even after functional specification of system becomes stable.

Identifying boundary objects

- *Heuristics for identifying boundary objects*
 - Identify **forms and windows** users needs to **enter data into system**
 - Identify **notices and messages** system uses to respond to user
 - Do not model visual aspects of interface with boundary objects (user mock-ups are better suited for that).
 - Always use user's terms for describing interfaces as opposed to terms from implementation technology.
- The terms used for describing boundary objects in analysis model should follow **user terminology**, even if it is tempting to use terms from the **implementation domain**.

Identifying control objects

- Control objects are responsible for **coordinating boundary and entity objects**.
- There is often a close relationship between **a use case and a control object**.
- A control object **is usually created at the beginning of a use case** and exist at its end.
- It is responsible for collecting **information from the boundary objects** and dispatching it to **entity objects**.
- For example, control objects describe the behavior associated with sequencing of **forms, undo and history queues, and dispatching** information in a distributed system.

Identifying control objects

- **Heuristics for identifying control objects**
 - Identify one **control object per use case** or more if use case is complex and if it can be divided into shorter flows of events.
 - Identify one control object per actor in the use case.
 - The life span of a control object should be extent of use case or the extent of a user session.
 - If it is difficult to identify beginning and end of a control object activation, **corresponding use case may not have a well-defined entry and exit condition**

Mapping use cases to objects

Modeling Interactions Between Objects: *Sequence Diagrams*

- A sequence diagram ties *use cases with objects*.
- It shows how behavior of a use case (or scenario) is *distributed among its participating objects*.
- Sequence diagrams are usually *not a good medium for communication with user*.
- They represent, however, another shift in perspective and allow developers to find missing objects or grey areas in the system specification.



Mapping use cases to objects

- **Heuristics for drawing sequence diagrams**
 - The **first column** should **correspond to the actor** who initiated the use case.
 - The **second column** should be an **boundary object** (that actor used to initiate use case).
 - The **third column** **should be the control object** that manages the rest of the use case.
 - **Control objects are created by boundary objects** initiating use cases.
 - **Boundary objects are created by control objects.**
 - **Entity objects are accessed by control and boundary objects.**
 - **Entity objects never access boundary or control objects**, this makes it easier to share entity objects across use cases.

Cont....

- *Identifying associations among objects*
- *Identifying object attributes*
- *Modeling nontrivial behavior with state charts*
- *Modeling generalization relationships*

Reviewing Analysis Model

- The analysis model is built **incrementally and iteratively**.
- The client, users, and developers examine the model for **correctness, consistency, completeness, and realism**.
- **Multiple reviews** should be done to ensure **high quality of requirements** and to allow space for learning requirements activity.
- For example, an **omission discovered during analysis** will lead to adding or extending a use case in the system specification, which may lead to eliciting more information from the user.

Reviewing Analysis Model

- Once analysis model **becomes stable** (i.e., when the number of **changes to the model are minimal and the scope of the changes localized**), the analysis model is **reviewed**,
 - first by the developers (i.e., internal reviews),
 - Then jointly by the developers and the client.
- The goal of the review is to make sure that system specification is **correct, complete, consistent, and realistic**.
- The review can be facilitated by a **checklist or a list of questions**.

Reviewing Analysis Model

- Below are example questions adapted from [Jacobson et al., 1999] and [Rumbaugh et al., 1991]. to ensure that **the model is correct**:
 - Is the glossary of entity objects **understandable by the user**?
 - Do **abstract classes correspond** to user-level concepts?
 - Are all descriptions in accordance with the **users' definitions**?
 - Do all **entity and boundary objects** have meaningful noun phrases as names?
 - Do all **use cases and control objects** have meaningful verb phrases as names?
 - Are all **error cases described** and handled?
 - Are the **start-up and the shut-down phases** of the system described?
 - Are the **administration functions** of the system described?

Reviewing Analysis Model

- The following questions should be asked to ensure that the model is **complete**:
 - For each object:
 - Is it needed by any use case?
 - In which use case is it **created? modified? destroyed?**
 - Can it be accessed from an boundary object?
 - For each attribute:
 - When is it set? What is its type? Should it be a qualifier?
 - For each association:
 - When is it traversed?
 - Why was the specific multiplicity chosen?
 - Can associations with one-to-many and many-to-many multiplicities be qualified?
 - For each control object:
 - Does it have the necessary associations to access the objects participating in its corresponding use case?

Reviewing Analysis Model

- The following questions should be asked to ensure that the model is **consistent**:
 - Are there multiple classes or use cases with the **same name**?
 - Do entities (e.g., use cases, classes, attributes) with **similar names denote similar phenomena**?
 - Are all entities described at the **same level of detail**?
 - Are there objects with **similar attributes and associations that are not in the same generalization hierarchy**?
- The following questions should be asked to ensure that the system described by the analysis **model is realistic**:
 - Are there any novel features in the system?
 - Were there any studies or prototypes built to ensure their feasibility?
 - Can the performance and reliability requirements be met? Were these requirements verified by any prototypes running on the selected hardware?

Managing Analysis

- *The primary challenge in managing the requirements in a project is to maintain consistency while using so **many resources**.*
- *In the end, the requirements analysis document should describe a single coherent **system understandable to a single person**.*
 - *Document template to document the results of analysis*
 - *Role assignment to analysis*
 - *Communication issues during analysis.*
 - *Management issues related to iterative and incremental nature of requirements*

Thank you for ur attention

Questions?